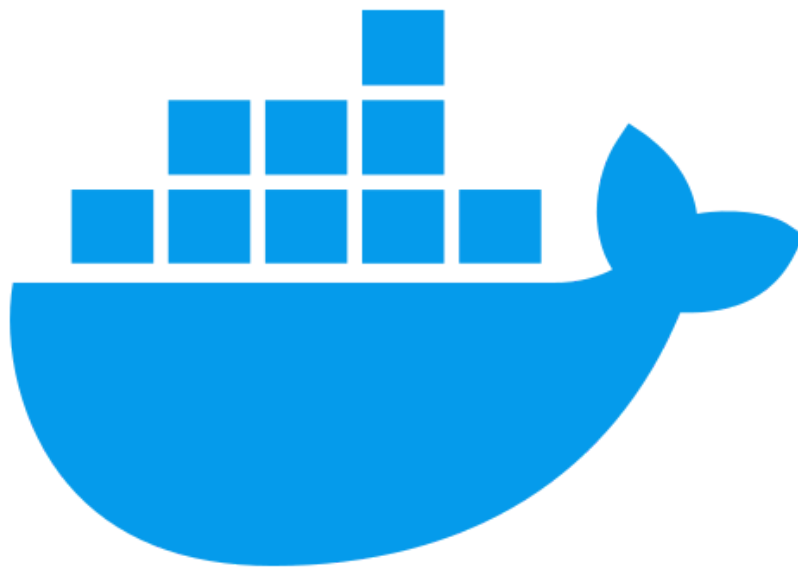

SWAP P4: SEGURIDAD (cortafuegos)

ÁLVARO GARCÍA ZAFRA



TIEMPO

TÍTULO ↕

DURACIÓN ↕

• SWAP P4

08:16:56

BÁSICAS

Tareas Básicas - B1. Preparación del Entorno de Trabajo

El primer paso ha sido llevar a cabo la creación de los directorios necesarios para almacenar los archivos de configuración de los servidores, del balanceador y de los certificados (he copiado los de la práctica anterior y les he cambiado el nombre).

Una vez hecho esto he creado el directorio que contendrá los scripts necesarios para la configuración de IPTABLES (en el directorio apache).

```
├── docker-compose.yml
├── P4-algarza04-apache
│   ├── algarza04-apache-ssl.conf
│   ├── DockerfileApache_algarza04
│   ├── P4-algarza04-iptables-web
│   └── web_algarza04
│       └── index.php
├── P4-algarza04-certificados
│   ├── certificado_algarza04.crt
│   └── certificado_algarza04.key
├── P4-algarza04-nginx
│   ├── algarza04-nginx-ssl.conf
│   ├── DockerfileNginx_algarza04
│   └── web_algarza04
│       └── index.php
```

Así quedó finalmente tras la creación de todos los archivos:

```
├── docker-compose.yml
├── P4-algarza04-apache
│   ├── algarza04-apache-ssl.conf
│   ├── DockerfileApache_algarza04
│   ├── P4-algarza04-iptables-web
│   │   ├── algarza04-iptables-web.sh
│   │   └── entrypoint.sh
│   └── web_algarza04
│       └── index.php
├── P4-algarza04-certificados
│   ├── certificado_algarza04.crt
│   └── certificado_algarza04.key
├── P4-algarza04-nginx
│   ├── algarza04-nginx-ssl.conf
│   ├── DockerfileNginx_algarza04
│   └── web_algarza04
│       └── index.php
```

Tareas Básicas - B2. Definición de Políticas de Seguridad en los Servidores Web

En esta parte hemos desarrollado el script “[algarza04-iptables-web.sh](#)” que establecerá las reglas del cortafuegos(IPTABLES) necesarias para los servidores web:

```
#!/bin/bash

# Establecemos una política de denegación implícita de todo el tráfico
# Descartamos cualquier paquete que no cumpla con una regla explícita
# el parámetro -P establece la política por defecto de las cadenas INPUT, OUTPUT y FORWARD
iptables -P INPUT DROP
iptables -P OUTPUT DROP
iptables -P FORWARD DROP

# Permitir que las respuestas iniciadas por localhost se acepten
# el parámetro -A agrega una regla
# -m state permite filtrar paquetes establecidos(ESTABLISHED) y relacionados(RELATED) a una conexión
# -j ACCEPT indica que se acepten los paquetes que cumplan lo anterior
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Permitir que localhost inicie y mantenga conexiones sin interrupción
# los parámetros de esta regla son los mismos que la anterior pero para OUTPUT
iptables -A OUTPUT -m state --state NEW,ESTABLISHED,RELATED -j ACCEPT

# Permitir el tráfico que el host se envía a sí mismo
# en cada uno añadimos una regla (INPUT/OUTPUT) para aceptar el tráfico del localhost(-i input, -o output)
iptables -A INPUT -i lo -j ACCEPT
iptables -A OUTPUT -o lo -j ACCEPT

# Permitir el tráfico TCP entrante al puerto 80 y 443 si proviene del balanceador
# Añadimos una regla para el protocolo TCP, solo si viene de 192.168.19.50
# -dport indica el puerto de destino
iptables -A INPUT -p tcp -s 192.168.10.50 --dport 80 -j ACCEPT
iptables -A INPUT -p tcp -s 192.168.10.50 --dport 443 -j ACCEPT
```

- Este script establecerá las siguientes reglas:
 - Rechazamos todo el tráfico que no tengamos explícitamente permitido.
 - Permitimos el tráfico iniciado por el host, a nivel de red y local .
 - Permitimos el tráfico por los puertos 80 y 443 si proviene de la dirección del balanceador de carga.

El objetivo principal de este script es garantizar la seguridad en el tráfico de la red, ya que se encargará de permitir únicamente el tráfico necesario para el funcionamiento de nuestro sistema. Para esto se establece una política de denegación, eliminando todo el tráfico que no cumpla con las normas establecidas explícitamente en nuestra configuración.

Tareas Básicas - B3. Configuración de Servidores Web con IPTABLES

Una vez ya tenemos configurado el script IPTABLES vamos a pasar a integrarlo en el Dockerfile de apache.

Para ello hemos creado en primer lugar el script “[entrypoint.sh](#)”, el cuál añadiremos posteriormente al Dockerfile. Este script se encargará de ejecutar la configuración IPTABLES que hemos creado para limitar el tráfico del contenedor y posteriormente lanzará dicho contenedor.

```
#!/bin/bash

# Ejecuta el script de iptables
./algarza04-iptables-web.sh

# Luego, ejecuta el comando principal del contenedor
exec "$@"
```

Una vez ya tenemos el script anterior vamos a modificar el Dockerfile para añadir dicho script como ENTRYPOINT(comando que se ejecutará cuando se inicie el contenedor e indicará cómo se debe iniciar este) y añadiremos algunos parámetros más.

```
# El contenedor se crea a partir de la última imagen Debian
FROM debian:latest

# Indica que no se solicitará interacción con el usuario durante la instalación de paquetes
ENV DEBIAN_FRONTEND=noninteractive

# Instalar las dependencias necesarias
# (servidor web Apache, PHP, paquete de PHP para Apache, la herramienta ping e iptables, además limpia la caché)
RUN apt-get update && apt-get install -y apache2 php libapache2-mod-php iputils-ping iptables && \
    rm -rf /var/lib/apt/lists/*

# Copia el .php al directorio de Apache que contiene los ficheros html
COPY P4-algarza04-apache/web_algarza04/index.php /var/www/html/

# Copiar certificado y clave privada
COPY P4-algarza04-certificados/certificado_algarza04.crt /etc/apache2/ssl/certificado_algarza04.crt
COPY P4-algarza04-certificados/certificado_algarza04.key /etc/apache2/ssl/certificado_algarza04.key

# Incluir la configuración SSL
COPY P4-algarza04-apache/algarza04-apache-ssl.conf /etc/apache2/sites-available/algarza04-apache-ssl.conf

# Copiar script de entrada entrypoint y script de reglas iptables
COPY P4-algarza04-apache/P4-algarza04-iptables-web/entrypoint.sh /entrypoint.sh
COPY P4-algarza04-apache/P4-algarza04-iptables-web/algarza04-iptables-web.sh /algarza04-iptables-web.sh

# Instalar módulo y habilitar sitio SSL por defecto y crear directorio para certificados
# Configurar los permisos adecuados para el certificado
# Dar permisos de ejecución a los scripts iptables
RUN a2enmod ssl && a2ensite default-ssl && \
    chmod 600 /etc/apache2/ssl/certificado_algarza04.crt && \
    chmod +x /entrypoint.sh /algarza04-iptables-web.sh

# Configurar el script de entrada
ENTRYPOINT ["/entrypoint.sh"]

# Exponer el puerto HTTP y HTTPS
EXPOSE 80
EXPOSE 443

# Inicia el servidor Apache en primer plano y mantiene el contenedor en ejecución(tail -f /dev/null)
CMD apache2ctl -D FOREGROUND && tail -f /dev/null
```

- El resto de parámetros añadidos son:
 - Instalar iptables.
 - Copiar el entrypoint y las reglas iptables.
 - Dar permisos de ejecución a dichos script.

Como he hecho en las prácticas anteriores voy a comprobar que el Dockerfile es correcto antes de pasar a ejecutarlo en el docker-compose, de esta manera puedo saber si este tiene algún error que no me dejará levantar los contenedores correctamente.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker build -t apache-p4-prueba -f P4-algarza04-apache/DockerfileApache_algarza04 .
[+] Building 2.6s (14/14) FINISHED
docker:default
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker images
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
apache-p4-prueba	latest	5102f0a1198b	33 seconds ago	269MB

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker run -d -p 8090:443 -p 9090:80 --name apache-p4 apache-p4-prueba:latest
d546bda3141ca02ad3243560b5b5b6cb500e74c3dcd5072f59209741a2831ef8
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
d546bda3141c	apache-p4-prueba:latest	"/entrypoint.sh /bin..."	5 seconds ago	Up 5 seconds	0.0.0.0:8090->80/tcp, [::]:8090->80/tcp, 0.0.0.0:8090->443/tcp, [::]:8090->443/tcp

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker exec -it apache-p4 /bin/bash
root@d546bda3141c:/# ls
algarza04-iptables-web.sh  boot  entrypoint.sh  home  lib64  mnt  proc  run  srv  tmp  var
bin                        dev   etc             lib   media  opt  root  sbin sys  usr
```

Como podemos ver se ha levantado correctamente y se puede ver también que se han copiado los dos scripts nuevos que hemos creado. Por último voy a comprobar que el cortafuegos está funcionando correctamente y podré dar por válida la configuración apache.

En un principio esto no me funcionaba porque no tenía el modo root activado, para esto volví a hacer el run añadiendo `--cap-add=NET_ADMIN`, de esta manera tiene permisos de admin:

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker run --cap-add=NET_ADMIN -d -p 8090:443 -p 9090:80 --name apache-p4 apache-p4-prueba:latest
8079f8f7e9d0e8b0e7e5c09bc238e48db7d8d899dcd1db5461ef251076f11e4
```

Una vez hecho esto ya pude comprobar que se habían aplicado las normas iptables con el comando "iptables -L":

```
root@8079f8f7e9d0:/# iptables -L
```

target	prot	opt	source	destination	state
ACCEPT	all	--	anywhere	anywhere	RELATED,ESTABLISHED
ACCEPT	all	--	anywhere	anywhere	
ACCEPT	tcp	--	192.168.10.50	anywhere	tcp dpt:http
ACCEPT	tcp	--	192.168.10.50	anywhere	tcp dpt:https

target	prot	opt	source	destination
Chain FORWARD (policy DROP)				
target	prot	opt	source	destination

target	prot	opt	source	destination	state
ACCEPT	all	--	anywhere	anywhere	NEW,RELATED,ESTABLISHED
ACCEPT	all	--	anywhere	anywhere	

Con esto podemos dar por válido todo lo creado hasta el momento.

Tareas Básicas - B4. Docker Compose y Despliegue de la Granja Web con IPTABLES

Una vez ya hemos configurado todo lo anterior solo nos falta modificar el DockerCompose para que incluya en los contenedores de Apache y de Nginx la funcionalidad de IPTABLES.

Para esto he copiado el docker-compose de la práctica anterior y lo he adaptado a lo que se necesita en esta práctica, en primer lugar he cambiado los nombres de las carpetas (de p3 a p4) y he añadido un nuevo parámetro (“cap_add: NET_ADMIN”, como ya he explicado anteriormente esto le dará permisos de administrador en el contenedor, esto servirá para poder modificar la configuración de red y aplicar nuestras reglas iptables), a parte de esto no he tenido que realizar ningún otro cambio. A continuación podemos ver uno de los contenedores y el balanceador:

```
# inicialización del contenedor web1:
# -es creado a partir del Dockerfile de apache
# -se le asigna su nombre e imagen
# -se asignan un puerto HTTPS y HTTP
# -se asignan los volúmenes (dir web que posee el php y dir que posee los certificados)
# -se asignan las redes (red_web y red_servicios) con su ip estática correspondiente
# -se asigna la capacidad de administrar la red
D Run Service
web1:
  build:
    context: .
    dockerfile: P4-algarza04-apache/DockerfileApache_algarza04
  image: algarza04-apache-image:p4
  container_name: web1
  ports:
    - 8081:443
    - 9091:80
  volumes:
    - ./P4-algarza04-apache/web_algarza04:/var/www/html
    - ./P4-algarza04-certificados:/etc/apache2/ssl/
  networks:
    red_web:
      ipv4_address: 192.168.10.2
    red_servicios:
      ipv4_address: 192.168.20.2
  cap_add:
    - NET_ADMIN
```

```
# inicialización del balanceador-nginx-ssl:
# -se crea a partir del Dockerfile de nginx
# -se le asigna su nombre e imagen
# -se asigna un puerto (asociado al puerto HTTPS del contenedor, ya que servirá las peticiones de forma segura)
# -se asignan los volúmenes (dir que posee los certificados y dir que posee el archivo de configuración del balanceador)
# -se asigna la red_web con su ip estática correspondiente
# -se asigna la dependencia de los contenedores web1-8 (ya que el balanceador no puede iniciar sin ellos inicializados)
D Run Service
balanceador-nginx-ssl:
  build:
    context: .
    dockerfile: P4-algarza04-nginx/DockerfileNginx_algarza04
  image: algarza04-nginx-image:p4
  container_name: balanceador-nginx
  ports:
    - 8090:443
    - 9090:80
  volumes:
    - ./P4-algarza04-certificados:/etc/nginx/ssl/
    - ./P4-algarza04-nginx/algarza04-nginx-ssl.conf:/etc/nginx/nginx.conf
  networks:
    red_web:
      ipv4_address: 192.168.10.50
  depends_on:
    - web1
    - web2
```

Tareas Básicas - B5. Verificación del escenario con IPTABLES

Por último, para terminar con las tareas básicas vamos a pasar al despliegue y verificación del escenario.

Ejecución del docker-compose:

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker-compose up -d
Creating network "red_web" with the default driver
Creating network "red_servicios" with the default driver
Creating web5 ... done
Creating web2 ... done
Creating web1 ... done
Creating web8 ... done
Creating web3 ... done
Creating web4 ... done
Creating web6 ... done
Creating web7 ... done
Creating balanceador-nginx ... done
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
6fa48823a994	algarza04-nginx-image:p4	"/docker-entrypoint..."	3 seconds ago	Up 2 seconds	0.0.0.0:9090->80/tcp, [::]:9090->80/tcp,
0.0.0.0:8090->443/tcp, [::]:8090->443/tcp	balanceador-nginx	"/entrypoint.sh /bin..."	3 seconds ago	Up 3 seconds	0.0.0.0:9093->80/tcp, [::]:9093->80/tcp,
9c26ef9da86e	algarza04-apache-image:p4	web3	3 seconds ago	Up 3 seconds	0.0.0.0:9096->80/tcp, [::]:9096->80/tcp,
0.0.0.0:8083->443/tcp, [::]:8083->443/tcp	web6	3 seconds ago	Up 3 seconds	0.0.0.0:9098->80/tcp, [::]:9098->80/tcp,	
4e6db5dd6c00	algarza04-apache-image:p4	web8	3 seconds ago	Up 3 seconds	0.0.0.0:9094->80/tcp, [::]:9094->80/tcp,
0.0.0.0:8086->443/tcp, [::]:8086->443/tcp	web4	3 seconds ago	Up 3 seconds	0.0.0.0:9097->80/tcp, [::]:9097->80/tcp,	
1839b044cb31	algarza04-apache-image:p4	web7	3 seconds ago	Up 3 seconds	0.0.0.0:9095->80/tcp, [::]:9095->80/tcp,
0.0.0.0:8088->443/tcp, [::]:8088->443/tcp	web5	3 seconds ago	Up 3 seconds	0.0.0.0:9092->80/tcp, [::]:9092->80/tcp,	
a0e087eff388	algarza04-apache-image:p4	web2	3 seconds ago	Up 3 seconds	0.0.0.0:9091->80/tcp, [::]:9091->80/tcp,
0.0.0.0:8084->443/tcp, [::]:8084->443/tcp	web1				
26fbc3a63621	algarza04-apache-image:p4				
0.0.0.0:8087->443/tcp, [::]:8087->443/tcp					
c395f12d8c82	algarza04-apache-image:p4				
0.0.0.0:8085->443/tcp, [::]:8085->443/tcp					
293f9b295e55	algarza04-apache-image:p4				
0.0.0.0:8082->443/tcp, [::]:8082->443/tcp					
438c76b761b9	algarza04-apache-image:p4				
0.0.0.0:8081->443/tcp, [::]:8081->443/tcp					

Una vez hecho esto pasaremos a comprobar el funcionamiento del balanceador:

Primeros 4 contenedores a los que nos manda el balanceador con https

< > C

No es seguro

https://localhost:8090

SWAP - "Álvaro García Zafra"

Dirección IP del servidor: 192.168.10.2

< > C

No es seguro

https://localhost:8090

SWAP - "Álvaro García Zafra"

Dirección IP del servidor: 192.168.10.3

< > C

No es seguro

https://localhost:8090

SWAP - "Álvaro García Zafra"

Dirección IP del servidor: 192.168.10.4

< > C

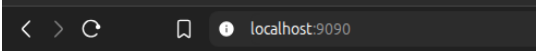
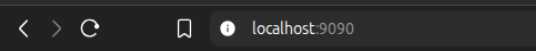
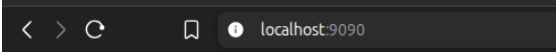
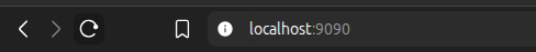
No es seguro

https://localhost:8090

SWAP - "Álvaro García Zafra"

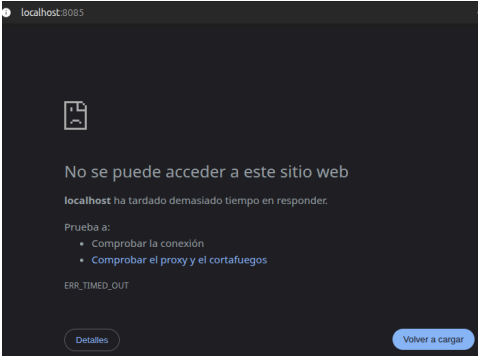
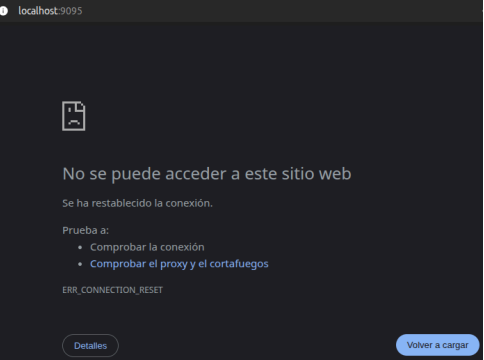
Dirección IP del servidor: 192.168.10.5

Primeros 4 contenedores a los que nos manda el balanceador con http

 SWAP - "Álvaro García Zafra" Dirección IP del servidor: 192.168.10.2	 SWAP - "Álvaro García Zafra" Dirección IP del servidor: 192.168.10.3
 SWAP - "Álvaro García Zafra" Dirección IP del servidor: 192.168.10.4	 SWAP - "Álvaro García Zafra" Dirección IP del servidor: 192.168.10.5

Solo he puesto los 4 primeros ya que el funcionamiento es el mismo, irá mostrando los servidores que quedan en orden, es decir, 6-7-8-9

Por otro lado podemos ver que al hacer la petición directamente al contenedor no podemos acceder a él, lo que significa que la configuración se está aplicando de manera correcta. A continuación podemos ver un ejemplo de esto (he puesto solo un ejemplo de cada uno ya que da el mismo resultado para todos los contenedores):

https	http
	

Ejemplo de iptables en uno de los contenedores (en el siguiente ejemplo podemos ver la configuración de iptables de web4, como podemos ver la configuración es la de nuestro script):

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker exec -it web4 /bin/bash
root@786305cc1ad6:/# iptables -L
Chain INPUT (policy DROP)
target     prot opt source                destination            state RELATED,ESTABLISHED
ACCEPT    all  --  anywhere              anywhere
ACCEPT    all  --  anywhere              anywhere
ACCEPT    tcp  --  balanceador-nginx.red_web anywhere              tcp dpt:http
ACCEPT    tcp  --  balanceador-nginx.red_web anywhere              tcp dpt:https

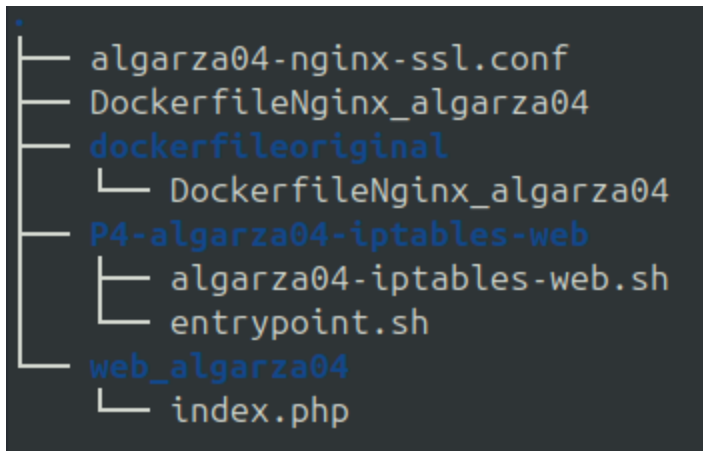
Chain FORWARD (policy DROP)
target     prot opt source                destination

Chain OUTPUT (policy DROP)
target     prot opt source                destination
ACCEPT    all  --  anywhere              anywhere
ACCEPT    all  --  anywhere              anywhere
```


AVANZADAS

Tareas Avanzadas - A1. Políticas avanzadas en el balanceador

En esta tarea vamos a implementar una serie de políticas en el balanceador de carga, para esto he creado un directorio que contendrá los scripts de configuración, he añadido “cap_add: net_admin” al dockercompose y he modificado el Dockerfile de Nginx(descargar iptables, copiar los scripts y darles permisos de ejecución y añadir un entryptoint) para que se aplique dicha configuración.



A continuación podemos ver las reglas que le he añadido al script:

```
#!/bin/bash

# Limitar conexiones simultáneas para cada IP por el puerto 80 (HTTP) y 443 (HTTPS)
iptables -A INPUT -p tcp --syn --dport 80 -m connlimit --connlimit-above 10 --connlimit-mask 32 -j REJECT --reject-with tcp-reset
iptables -A INPUT -p tcp --syn --dport 443 -m connlimit --connlimit-above 10 --connlimit-mask 32 -j REJECT --reject-with tcp-reset

# Permitir solo puertos necesarios
iptables -P INPUT DROP
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

# Bloquear escaneo de puertos (port scanning)
iptables -N PORTSCAN
iptables -A PORTSCAN -p tcp --tcp-flags SYN,ACK,FIN,RST RST -m limit --limit 1/s --limit-burst 2 -j RETURN
iptables -A PORTSCAN -j DROP
iptables -A INPUT -p tcp --tcp-flags SYN,ACK,FIN,RST RST -j PORTSCAN
```

- **Permitir solo puertos necesarios:** esta primera parte es similar a la realizada para los contenedores apache. Se establece una política de bloqueo en la cadena INPUT, aceptando solo los que lleguen por el puerto 80 y 443, acepta también el tráfico del localhost y también que las respuestas que inicie este se acepten.
- **Limitar conexiones simultáneas:** añadimos una regla al tráfico entrante del protocolo TCP, para los puertos 80 y 443. El funcionamiento será el siguiente, evitamos que se cuenten los paquetes que ya han establecido una conexión (syn), limitamos el número de

conexiones simultáneas a 10 por ip (above indica el número y mask la máscara a que se aplica, en este caso es por ip, en el caso de que no se le añade se pondría por defecto máscara 32), por último tenemos un comando que se encarga de rechazar los paquetes y mandar un tcp-reset para terminar la conexión (-j REJECT...).

- **Bloquear escaneo de puertos:** añadimos una nueva cadena a la que se redirigirá el tráfico. Dentro de la cadena añadimos una regla que evaluará los paquetes TCP que tengan RST (RCP Reset) activado, limitaremos la frecuencia de estos paquetes a 1 por segundo, sin embargo se permitirá inicialmente una rafaga de 2 paquetes sin que sean bloqueados, si el tráfico cumple con esta regla continuará sin bloquearse, en caso contrario se manda a DROP y quedará bloqueado. Finalmente añadimos una última regla que redirigirá el tráfico TCP RST de INPUT a la cadena PORTSCAN para ser evaluado.

PRUEBAS DE FUNCIONAMIENTO

El primer paso es comprobar que se sigue levantando correctamente el servicio y que las reglas se aplican correctamente (como podemos ver en las capturas se está levantando el balanceador, las reglas que se han cargado son las que le hemos añadido al script y al acceder a su web sigue funcionando correctamente):

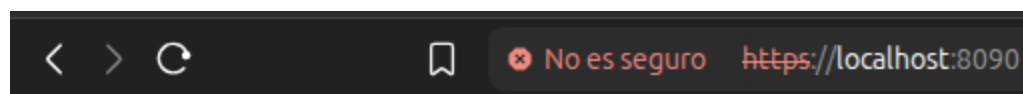
```
ff62d3e2786f  algarza04-nginx-image:p4  "/entrypoint.sh /bin..." 15 seconds ago  Up 14 seconds  0.0.0.0:9090->80/tcp, [::]:9090->80/tcp, 0.0.0.0:8090->443/tcp, [::]:8090->443/tcp  balanceador-nginx
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ docker exec -it balanceador-nginx /bin/bash
root@650e1ea2ddb:/# iptables -L -v -n --line-numbers
Chain INPUT (policy DROP 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 REJECT     6    --  *      *        0.0.0.0/0            0.0.0.0/0            tcp dpt:80 flags:0x17/0x02 #conn src/32 > 10 reject-with tcp-reset
2      0      0 REJECT     6    --  *      *        0.0.0.0/0            0.0.0.0/0            tcp dpt:443 flags:0x17/0x02 #conn src/32 > 10 reject-with tcp-reset
3      0      0 PORTSCAN   6    --  *      *        0.0.0.0/0            0.0.0.0/0            tcp flags:0x17/0x04
4      0      0 ACCEPT     6    --  *      *        0.0.0.0/0            0.0.0.0/0            tcp dpt:80
5      0      0 ACCEPT     6    --  *      *        0.0.0.0/0            0.0.0.0/0            tcp dpt:443
6      0      0 ACCEPT     0    --  lo     *        0.0.0.0/0            0.0.0.0/0
7      0      0 ACCEPT     0    --  *      *        0.0.0.0/0            0.0.0.0/0            state RELATED,ESTABLISHED

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
num  pkts bytes target     prot opt in     out     source               destination

Chain PORTSCAN (1 references)
num  pkts bytes target     prot opt in     out     source               destination
1      0      0 RETURN     6    --  *      *        0.0.0.0/0            0.0.0.0/0            tcp flags:0x17/0x04 limit: avg 1/sec burst 2
2      0      0 DROP       0    --  *      *        0.0.0.0/0            0.0.0.0/0
```



SWAP - "Álvaro García Zafra"

Dirección IP del servidor: 192.168.10.2

Para comprobar que no solo se están cargando correctamente vamos a realizar las siguientes pruebas:

- Bloqueo de las conexiones ip: **`ab -n 100 -c 50 https://localhost:8090/`**

pkts	bytes	target	prot	opt	in	out	source	destination	
0	0	REJECT	6	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:80 flags:0x17/0x02 #conn src/32 > 10 reject-with tcp-reset
146	8760	REJECT	6	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:443 flags:0x17/0x02 #conn src/32 > 10 reject-with tcp-reset
0	0	ACCEPT	6	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:80
140	18898	ACCEPT	6	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp dpt:443
0	0	ACCEPT	0	--	lo	*	0.0.0.0/0	0.0.0.0/0	
51	5776	ACCEPT	0	--	*	*	0.0.0.0/0	0.0.0.0/0	state RELATED,ESTABLISHED
0	0	PORTSCAN	6	--	*	*	0.0.0.0/0	0.0.0.0/0	tcp flags:0x17/0x04

Esta regla es correcta ya que podemos ver que le estamos mandando las solicitudes al puerto 443 y cuando supera el límite de 10 conexiones simultáneas está rechazando los paquetes(146 paquetes), podemos ver que también hay otros aceptados antes de llegar al límite de 10 (140 paquetes) y vemos que las conexiones válidas continúan sin ser bloqueadas (51 paquetes).

- Permitir solo los puertos necesarios: como podemos observar en la captura anterior está aceptando el tráfico que llega por el puerto 443, si hubiésemos mandado también al 80 pasaría lo mismo, por lo que está correctamente aplicado.
- Bloqueo de escaneo de puertos:

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ sudo nmap -sS -T4 --scan-delay 1ms -p 1-10000 192.168.10.50
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-05-17 11:44 CEST
Nmap scan report for 192.168.10.50
Host is up (0.00012s latency).
Not shown: 9998 filtered tcp ports (no-response)
PORT      STATE SERVICE
80/tcp    open  http
443/tcp   open  https
MAC Address: 3A:BA:6E:F1:1C:7B (Unknown)
Nmap done: 1 IP address (1 host up) scanned in 46.84 seconds
```

```
root@650e1ea2ddb:/# tcpdump -i any port 8090 or port 9090
tcpdump: data link type LINUX_SLL2
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on any, link-type LINUX_SLL2 (Linux cooked v2), snapshot length 262144 bytes
09:45:10.426857 eth0 In IP agz-ubu24.51665 > 650e1ea2ddb.9090: Flags [S], seq 3494086566, win 1024, options
], length 0
09:45:10.528816 eth0 In IP agz-ubu24.51667 > 650e1ea2ddb.9090: Flags [S], seq 3493955492, win 1024, options
], length 0
09:45:26.665150 eth0 In IP agz-ubu24.51665 > 650e1ea2ddb.8090: Flags [S], seq 3494086566, win 1024, options
], length 0
09:45:26.765643 eth0 In IP agz-ubu24.51667 > 650e1ea2ddb.8090: Flags [S], seq 3493955492, win 1024, options
], length 0
```

He entrado a la terminal de mi balanceador y he usado el comando que se ve en la captura, el cuál sirve para capturar en tiempo real el tráfico de red que llega o sale por los puertos 8090 y 9090, posteriormente desde mi terminal ejecute el escaneo de puertos a la ip de mi balanceador. Como podemos ver en el balanceador nos indica que los puertos no autorizados están filtrados y los dos que estamos usando para el tráfico están abiertos, lo cuál es lógico ya que si estuviesen close no se podría acceder a la web y si estuviesen filtered se descartaría el tráfico.

Tareas Avanzadas - A2. Configuración Avanzada de IPTABLES para DDoS

Para esta tarea vamos a configurar una serie de reglas para mitigar los ataques de Denegación de Servicio Distribuido (DDoS). Voy a implementar:

- Limitación de la tasa de conexiones nuevas por IP (está la desarrolle en el apartado anterior, por lo que en este apartado no volveré a hablar de ella).
- Uso de módulos para detectar y bloquear el tráfico anómalo rápidamente.
- Configuración de umbrales y reglas que identifiquen patrones de tráfico asociados a ataques comunes de DDoS.

Para esto al script anterior le hemos añadido las siguientes reglas:

```
# Si una IP establece más de 10 conexiones en 10 segundos, se la bloquea temporalmente.
iptables -A INPUT -p tcp --dport 80 -m recent --name ddos --update --seconds 10 --hitcount 10 -j DROP
iptables -A INPUT -p tcp --dport 80 -m recent --name ddos --set
iptables -A INPUT -p tcp --dport 443 -m recent --name ddos --update --seconds 10 --hitcount 10 -j DROP
iptables -A INPUT -p tcp --dport 443 -m recent --name ddos --set

# Protección personalizada contra DDoS
# Crear la nueva cadena
iptables -N DDOS-PROTECT

# Detectar IPs que abusan de nuevas conexiones
iptables -A DDOS-PROTECT -m recent --name badguy --rcheck --seconds 60 -j DROP

# Eliminar IPs que han hecho más de 15 conexiones en 60 segundos
iptables -A DDOS-PROTECT -m recent --name badguy --update --seconds 60 --hitcount 15 -j DROP

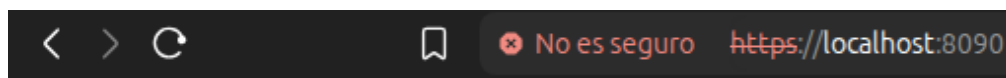
# Aplicar la cadena al tráfico entrante
iptables -A INPUT -p tcp --dport 80 -j DDOS-PROTECT
iptables -A INPUT -p tcp --dport 443 -j DDOS-PROTECT
```

- Uso del módulo recent (mantiene un registro de las IPs que intentan conectarse): para cada uno de los puertos (80 y 443) creamos dos reglas, ambas basan en crear una regla para el tráfico entrante que se aplique a los paquetes TCP, el funcionamiento de la primera se basará en que si alguna de las ips tiene al menos 10 intentos en los últimos 10 segundos se bloqueará esta ip de manera temporal (-update). Por otro lado la segunda registra cada IP que intente una conexión y la almacenará en la lista temporal ddos (-set, además cada puerto tendrá su propia lista).
- Configuración personalizada: para este conjunto de reglas hemos creado una nueva cadena, una vez está creada vamos añadiendo las reglas, las reglas que vamos a añadir tendrán el siguiente funcionamiento, las IPs que hagan más de 15 conexiones en 1 minuto serán bloqueadas durante 1 minuto (-rcheck se encarga de consultar si esta). Por otro lado, aplicamos esta cadena al tráfico entrante de los puertos 80 y 443, la cuál según las reglas anteriores decidirá si bloquear o aceptar las IPs.

PRUEBAS DE FUNCIONAMIENTO

Al igual que en el ejercicio anterior he comprobado que el servicio se sigue levantando correctamente y que las reglas se están aplicando correctamente

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
a9da7bc1668b	algarza04-nginx-image:pd	"/entrypoint.sh /bin..."	8 seconds ago	Up 8 seconds	0.0.0.0:9090->80/tcp, [::]:9090->80/tcp, 0.0.0.0:8090->443/tcp, [::]:8090->443/tcp	balanceador



SWAP - "Álvaro García Zafra"

Dirección IP del servidor: 192.168.10.2

```
root@a9da7bc1668b:/# iptables -L -v -n
Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination
  0      0 DROP      6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 DROP      6  --  *      *       0.0.0.0/0            0.0.0.0/0
 17 5243 DROP      6  --  *      *       0.0.0.0/0            0.0.0.0/0
 23 6221 DROP      6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 DDOS-PROTECT 6  --  *      *       0.0.0.0/0            0.0.0.0/0
 23 6221 DDOS-PROTECT 6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 REJECT    6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 REJECT    6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 PORTSCAN 6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 ACCEPT    6  --  *      *       0.0.0.0/0            0.0.0.0/0
 23 6221 ACCEPT   6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 ACCEPT    0  --  lo     *       0.0.0.0/0            0.0.0.0/0
  9 1124 ACCEPT    0  --  *      *       0.0.0.0/0            0.0.0.0/0
                                     state RELATED,ESTABLISHED

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain DDOS-PROTECT (2 references)
 pkts bytes target     prot opt in     out     source               destination
  0      0 DROP      0  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 DROP      0  --  *      *       0.0.0.0/0            0.0.0.0/0
                                     recent: CHECK seconds: 60 name: badguy side: source mask: 255.255.255.255
                                     recent: UPDATE seconds: 60 hit_count: 15 name: badguy side: source mask: 255.255.255.255

Chain PORTSCAN (1 references)
 pkts bytes target     prot opt in     out     source               destination
  0      0 RETURN    6  --  *      *       0.0.0.0/0            0.0.0.0/0
  0      0 DROP      0  --  *      *       0.0.0.0/0            0.0.0.0/0
                                     tcp flags:0x17/0x04 limit: avg 1/sec burst 2
```

Como podemos ver en las capturas anteriores el servicio sigue funcionando de manera correcta. Además al adentrarnos en la terminal del balanceador y poner el comando *iptables -L -v -n* (nos muestra las reglas que tenemos implementadas) se puede observar que las reglas que hemos introducido anteriormente se están aplicando correctamente al balanceador (la prueba práctica se encuentra en la Tarea Avanzada - A3). Además otra cosa que he podido observar es que al añadir estas reglas la web del balanceador tarda más en cargar, esto se debe a que comprueba si ha accedido más veces en los últimos segundos para bloquearla o no.

Tareas Avanzadas - A3. Simulación de ataques

Como he mencionado en la tarea anterior en esta última vamos a llevar a cabo una prueba para confirmar que las reglas iptables están realizando correctamente su función, para esto voy a llevar a cabo un ataque DDoS con el comando ApacheBench, a continuación podremos ver los dos ataques realizados y la salida de iptables:

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ ab -n 1000 -c 100 https://localhost:8090/
This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
apr_pollset_poll: The timeout specified has expired (70007)
Total of 1 requests completed
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P4$ ab -n 1000 -c 100 http://localhost:9090/

This is ApacheBench, Version 2.3 <$Revision: 1903618 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
apr_pollset_poll: The timeout specified has expired (70007)
```

```
root@9da7bc166bb:/# iptables -L -v -n
Chain INPUT (policy DROP 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination
 68 17693 DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 recent: UPDATE seconds: 10 hit_count: 10 name: ddos side: source mask: 255.255.255.255
 33 10301 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 recent: SET name: ddos side: source mask: 255.255.255.255
1228 103K DROP 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443 recent: UPDATE seconds: 10 hit_count: 10 name: ddos side: source mask: 255.255.255.255
 47 11853 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443 recent: SET name: ddos side: source mask: 255.255.255.255
 33 10301 DDOS-PROTECT 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
 47 11853 DDOS-PROTECT 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443
 0 0 REJECT 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80 flags:0x17/0x02 #conn src/32 > 10 reject-with tcp-reset
 0 0 REJECT 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443 flags:0x17/0x02 #conn src/32 > 10 reject-with tcp-reset
 1 40 PORTSCAN 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp flags:0x17/0x04
 33 10301 ACCEPT 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:80
 47 11853 ACCEPT 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp dpt:443
 0 0 ACCEPT 0 -- lo * 0.0.0.0/0 0.0.0.0/0
 52 6497 ACCEPT 0 -- * * 0.0.0.0/0 0.0.0.0/0 state RELATED,ESTABLISHED

Chain FORWARD (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination

Chain OUTPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target prot opt in out source destination

Chain DDOS-PROTECT (2 references)
 pkts bytes target prot opt in out source destination
 0 0 DROP 0 -- * * 0.0.0.0/0 0.0.0.0/0 recent: CHECK seconds: 60 name: badguy side: source mask: 255.255.255.255
 0 0 DROP 0 -- * * 0.0.0.0/0 0.0.0.0/0 recent: UPDATE seconds: 60 hit_count: 15 name: badguy side: source mask: 255.255.255.255

Chain PORTSCAN (1 references)
 pkts bytes target prot opt in out source destination
 1 40 RETURN 6 -- * * 0.0.0.0/0 0.0.0.0/0 tcp flags:0x17/0x04 limit: avg 1/sec burst 2
 0 0 DROP 0 -- * * 0.0.0.0/0 0.0.0.0/0
```

Como podemos observar las reglas parecen estar funcionando correctamente, ya que podemos observar que las de si reciben 10 peticiones en los últimos 10 segundos están bloqueando correctamente la conexión, la que no podemos apreciar su funcionamiento es la que hemos creado personalizadas de detectar las que abusan de nuevas conexiones y las que han hecho más de 15 conexiones en 60 segundos, esto se debe a que hay reglas anteriores que ya están bloqueando las conexiones y no dejan ninguna que cumpla esta regla.

ANÁLISIS PROPUESTA IA

Enlace al chat: <https://chatgpt.com/share/682c59d7-a29c-800d-a771-40b16fcbc986>

Resumen y opinión de su uso

Por último voy a hablar un poco de como me he ayudado de la IA para la realización de esta práctica.

En primer lugar me ha servido para saber llevar a cabo las pruebas necesarias para comprobar si mis scripts estaban bien creados, indicandome los comandos necesarios para ver que estas estaban bien aplicadas y que estaban bloqueando correctamente las conexiones.

También me ha ayudado a saber cuáles eran los parámetros que debía añadirle al balanceador para poder aplicarle las reglas y para qué servía cada uno de ellos. No obstante en esta parte me ha indicado reglas que no le estaba pidiendo o que no era exactamente lo que le pedía, o en otros casos me ha indicado algunas reglas que no estaban correctamente implementadas, un ejemplo de esto ha sido para la avanzada 2, en el que las reglas que me estaba dando estaban impidiendo que el balanceador me devolviera las páginas web de los contenedores, esto lo conseguí arreglar borrando algunas reglas que hacían que se produjera esto, o en otro momento que estaba creando una nueva cadena (SCAN) pero no le daba uso.

De esta manera a lo largo de esta práctica, al igual que con las anteriores, estoy aprendiendo en mayor medida a cómo sacarle el máximo provecho a esta herramienta, lo cual lo estoy consiguiendo sin mandarle los enunciados tal cuál y explicando que necesito y el contexto en que me encuentro, también leyendo y comprendiendo la respuesta que le da a mi pregunta para ver si es exactamente lo que estoy buscando.